Object Oriented Programming

Mihai Dascălu

Programming Style Brief and Documentation

Appropriate Comments

Naming Conventions

Proper Indentation and Spacing Lines

Block Styles

Appropriate Comments

/* Include a summary at the beginning of the program to explain what the program does, its key features, its supporting data structures, and any unique techniques */

// Useful comment

JavaDoc - https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html

@author (classes and interfaces only, required)

@version (classes and interfaces only, required)

@param (methods and constructors only)

@return (methods only)

@exception (@throws is a synonym added in Javadoc 1.2)

@since

@deprecated

Naming Conventions

- Choose meaningful and descriptive names
- Variables and method names:
  - Use lowercase
  - If the name consists of several words:
    - Concatenate all in one
    - Use lowercase for the first word
    - Capitalize the first letter of each subsequent word in the name
  - e.g., the variables radius and area and the method computeArea
- Class names:
  - Capitalize the first letter of each word in the name
  - e.g., the class name ComputeArea
- Constants:
  - Capitalize all letters in constants, and use underscores to connect words
  - e.g., the constant PI and MAX_VALUE

Proper Indentation and Spacing

- Indentation
  - Indent two spaces (or \t in Intellij / Eclipse / NetBeans)
- Spacing
  - Use blank line to separate segments of the code

Block Styles
Next-line & End-of-line

```
public class Test
{
  public static void main(String[] args)
  {
    System.out.println("Block Styles");
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Block Styles");
  }
}
```

Programming Errors

- Syntax Errors
  - Detected by the compiler
- Runtime Errors
  - Causes the program to abort
- Logic Errors
  - Produces incorrect result

Syntax Errors

```
public class ShowSyntaxErrors {
  public static void main(String[] args) {
    i = 30;
    System.out.println(i + 4);
  }
}
```

Runtime Errors

```
public class ShowRuntimeErrors {
  public static void main(String[] args) {
    int i = 1 / 0;
  }
}
```

Logic Errors

```
public class ShowLogicErrors {
  public static void main(String[] args) {
    // Prompt the user to enter a number
    String input = JOptionPane.showInputDialog(null,
      "Please enter an integer:",
      "ShowLogicErrors", JOptionPane.QUESTION_MESSAGE);

    int number = Integer.parseInt(input);
    // Display the result
    System.out.println("The number is between 1 and 100, " +
      "inclusively? " + ((1 < number) && (number < 100)));
    System.exit(0);
  }
}
```

Debugging

- *Bugs* = logic errors
- *Debugging* = the process of finding and correcting errors
- Common approach -> use a combination of methods to narrow down where the bug is located
  - Hand-trace the program
  - Insert print statements
  - Might work for short, simple programs
- For large, complex programs, the most effective approach for debugging is to use a **debugger**

Debugger

- A program that facilitates debugging & used to:
  - Execute a single statement at a time
  - Trace into or stepping over a method
  - Set breakpoints
  - Display variables
  - Display call stack
  - Modify variables
- https://www.jetbrains.com/help/idea/debugging-code.html#general-procedure
- https://www.eclipse.org/community/eclipse_newsletter/2017/june/article1.php
- https://netbeans.org/features/java/debugger.html

Intellij

- Valuable plugins
  - Copilot (AI pair programmer)
    - https://github.com/features/copilot/
  - Sonarlint Static code review: https://www.sonarlint.org/intellij
  - Key Promoter X – remember shortcuts
  - Rainbow Brackets – pretty colors
  - CSV
  - .ignore

Elementary programming

Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("Area for the circle of R=" +
      radius + " is " + area);
  }
}
```

allocate memory for radius

radius: no value

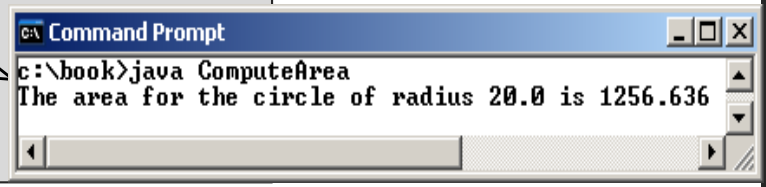
area: no value

radius: 20

area: 1256.636

compute area and assign it to variable area

print a message to the console



Reading Input from the Console

- Create a Scanner object
  - `Scanner input = new Scanner(System.in);`
- Use the methods next(), nextByte(), nextInt(), … to obtain to a String, byte, int, … value
- For example:

```
System.out.print("Enter a double value:");
Scanner input = new Scanner(System.in);
double d = input.nextDouble();
```

Identifiers

- A sequence of characters
  - Letters
  - Digits
  - Underscores (_)
  - Dollar signs ($)
- Must start with a letter, an underscore (_), or a dollar sign ($) & cannot start with a digit
  - An identifier cannot be a reserved word (e.g., true, false, or null)
- Can be of any length

Declaring Variables

```
int x;          // Declare x to be an
                // integer variable;
double radius; // Declare radius to
                // be a double variable;
char a;         // Declare a to be a
                // character variable;
```

Assignment Statements

```
x = 1;              // Assign 1 to x;

radius = 1.0;       // Assign 1.0 to radius;

a = 'A’;            // Assign 'A' to a;
```

Constants

```
final datatype CONSTANTNAME = VALUE;
```

```
final double PI = 3.14159;
final int SIZE = 3;
```

Primitive Data Types & Operators

byte: 8-bit signed [-128; 127]

short: 16-bit signed [-32,768; 32,767]

int: 32-bit signed [-2,147,483,648; 2,147,483,647] generally the default choice

long: 64-bit signed [-9,223,372,036,854,775,808; 9,223,372,036,854,775,807] exceeds int

float: single-precision 32-bit IEEE 754 floating point; should never be used for precise values, such as currency (java.math.BigDecimal)

double: double-precision 64-bit IEEE 754 floating point; usually the default choice for decimal values

Primitive Data Types & Operators

boolean: true and false; its "size" is not precisely defined

char: 16-bit Unicode character ['\u0000' (or 0); '\uffff' (or 65,535]

+, -, *, /, %

Shortcut assignment with =

++, -- (pre and post)

Default Values

Data Type	Default Value (for fields)
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
String (or any object)	null
boolean	false

Numeric Type Conversion

- Consider the following statements:
  - byte i = 100;
  - long k = i * 3 + 4;
  - double d = i * 3.1 + k / 2;
- Java automatically converts the operand based on the following rules:
  1. If one of the operands is *double*, the other is converted into *double*
  2. Otherwise, if one of the operands is *float*, the other is converted into *float*
  3. Otherwise, if one of the operands is *long*, the other is converted into *long*
  4. Otherwise, both operands are converted into *int*

Type Casting

- Implicit casting
  - double d = 3; (type widening)
- Explicit casting
  - int i = (int) 3.0; (type narrowing)
  - int i = (int) 3.9; (Fraction part is truncated)
- What is wrong? int x = 5 / 2.0;
  - double avg = 1/2 * (a+b);

range increases



byte, short, int, long, float, double

Character Data Type

```
char letter = 'A'; (ASCII)

char numChar = '4'; (ASCII)

char letter = '\u0041'; (Unicode)

char numChar = '\u0034'; (Unicode)
```



- NOTE: The increment and decrement operators can also be used on char variables to get the next or preceding Unicode character
  - char ch = 'a';
  - System.out.println(++ch);

Escape Sequences for Special Characters

Description	Escape Sequence	Unicode
Backspace	\b	\u0008
Tab	\t	\u0009
Linefeed	\n	\u000A
Carriage return	\r	\u000D
Backslash	\\	\u005C
Single Quote	\'	\u0027
Double Quote	\"	\u0022

The String Type

- Char type represents only one character. To represent a string of characters:

```
String message = "Welcome to Java";
```

- A predefined class in the Java library
- Not a primitive type, but a reference type

```
// Three strings are concatenated
  String message = "Welcome " + "to " + "Java";

// String Chapter is concatenated with number 2
  String s = "Chapter" + 2; // s becomes Chapter2

// String Supplement is concatenated with character B
  String s1 = "Supplement" + 'B'; // s1 becomes SupplementB
```

Converting Strings to Integers or Doubles

- To convert a String into an int value - static parseInt method in the Integer class:

  int intValue = Integer.parseInt(intString);

  where intString is a numeric string such as “123”

- To convert a string into a double value - static parseDouble method in the Double class:

  double doubleValue
  =Double.parseDouble(doubleString);

The `boolean` Type and Operators

- Often in a program you need to compare two values, such as whether i is greater than j =>
  - Boolean value: true or false
  - boolean b = (1 > 2);

Operator	Name
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
!=	not equal to

if Statements

```
if (boolean-expression) {
        statements;
}
//else {
//      statements-for-the-false-case;
//}
```

Multiple Alternative if Statements

- The else clause matches the most recent if clause in the same block.

```
if (score >= 90.0)
  grade = 'A';
else
  if (score >= 80.0)
    grade = 'B';
  else
    if (score >= 70.0)
      grade = 'C';
    else
      if (score >= 60.0)
        grade = 'D';
      else
        grade = 'F';
```

Equivalent

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

```
int i = 1;
int j = 2;
int k = 3;
if (i > j)
  if (i > k)
    System.out.println("A");
else
    System.out.println("B");
```

Equivalent

```
int i = 1;
int j = 2;
int k = 3;
if (i > j)
  if (i > k)
    System.out.println("A");
  else
    System.out.println("B");
```

Common Errors

- Adding a semicolon at the end of an if clause is a common mistake

```
if (radius >= 0);
{
  area = radius*radius*PI;
  System.out.println("The area for the circle of
  radius " +    radius + " is " + area);
}
```

Wrong

- This mistake is hard to find because it is not a compilation error or a runtime error, it is a logic error
- This error often occurs when you use the next-line block style

Efficiency

```
if (number % 2 == 0)
  even = true;
else
  even = false;
```

Equivalent

```
boolean even
  = number % 2 == 0;
```

```
if (even == true)
  System.out.println
   ("It is even.");
```

Equivalent

```
if (even)
  System.out.println
   ("It is even.");
```

Logical Operator

Operator	Name
!	not
&&	and
||	or
^	exclusive or
&	bitwise and
|	bitwise or

```
x=1: (1 == x) && ( 2 > x++)
```

switch Statement Rules (1)

Switch-expression must yield a value of char, byte, short, or int type (enclosed in parentheses)

```
switch (switch-expression) {
  case value1:  statement(s)1;
                break;
  case value2: statement(s)2;
                break;
  …
  case valueN: statement(s)N;
                break;
  default: something-default;
}
```

- value1, ..., and valueN must have the same data type as the value of the switch-expression
- Resulting statements in the case statement are executed *when* the value in the case statement matches the switch-expression
- value1, ..., and valueN are constant expressions => cannot contain variables in the expression, such as 1 + x

switch Statement Rules (2)

break is optional, but it **should** be used at the end of each case to terminate remainder statements

If the break statement is not present => the next case statement **will be executed**

The default case (optional) can be used to perform actions **when none** of the specified cases matches the switch-expression

```
switch (switch-expression) {
  case value1:  statement(s)1;
                break;
  case value2: statement(s)2;
                break;
  …
  case valueN: statement(s)N;
                break;
  default: something-default;
}
```

- The case statements are executed in **sequential order**
- The order of the cases (including the default case) is up to you!
- Good programming: follow the logical sequence of cases & place the default case at the end

Trace switch statement (1)

Suppose ch is 'a':

```
switch (ch) {
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

```
default: System.out.println("default"); ?
```

```
switch (ch) {
  case 'a':  System.out.println(ch);
             break;
  case 'b':  System.out.println(ch);
             break;
  case 'c':  System.out.println(ch);
             break;
}
```

Enhanced Switch (JDK 12+)

```
switch (switch-expression) {
  case value1 ->  statement(s)1;
  case value2, value 3 -> statement(s)2;
  case valueN -> statement(s)N;
  default -> statement(s)-for-default;
}
```

~~break;~~

Conditional Operator

```
(boolean-expression) ? exp1 : exp2
```

```
if (num % 2 == 0)
  System.out.println(num + “is even”);
else
  System.out.println(num + “is odd”);
```

```
System.out.println((num % 2 == 0)? num + ”is even” :
num + ”is odd”);
```

Formatting Output

- Use the printf statement
  `System.out.printf (format, items);`
- May consist of substrings and format specifiers (% how an item should be displayed)

Specifier	Output	Example
%b	a boolean value	true or false
%c	a character	'a'
%d	a decimal integer	200
%f	a floating-point number	45.460000
%e	a number in scientific notation	4.556000e+01
%s	a string	"Java is cool"

Formatting Example

```
int count = 5;

double amount = 45.56;

System.out.printf("|count is %d \t| amount is
%f\t|", count, amount);
```

Display: |count is 5 |amount is 45.560000 |

Operator Precedence

var++, var--

+, - (Unary plus and minus), ++var,--var

(type) Casting

! (Not)

*, /, % (Multiplication, division, and remainder)

+, - (Binary addition and subtraction)

<, <=, >, >= (Comparison)

==, !=; (Equality)

^ (Exclusive OR)

&& (Conditional AND) Short-circuit AND

|| (Conditional OR) Short-circuit OR

=, +=, -=, *=, /=, %= (Assignment operator)

Operator Precedence and Associativity

- The expression in the parentheses is evaluated first
- Parentheses can be nested
- When evaluating an expression without parentheses
  - The operators are applied according to the precedence rule and the associativity rule
- If operators with the same precedence are next to each other
  - Their associativity determines the order of evaluation
- All binary operators except assignment operators are *left-associative*
  - a – b + c – d is equivalent to ((a – b) + c) – d
- Assignment operators are *right-associative*
  - a = b += c = 5 is equivalent to a = (b += (c = 5))

Loops

Loops - Motivations

- Suppose you need to print a string (e.g., "Welcome to Java!") a hundred times
- Tedious to write the following statement 100 times:

```
System.out.println("Welcome to Java!");
```

100 times

```
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
...
...
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
```

Introducing while Loops

while (loop-continuation-condition) {

  // loop-body;

  Statement(s);

}

- Often the number of times a loop is executed is not predetermined => use an input value to signify the end of the loop (a *sentinel value*)

```
int count = 0;
while (count < 100) {
  System.out.println("Welcome to Java");
  count++;
}
```

Caution

- **Do not use floating-point values** for **equality** checking in a loop control

- Consider the following code for computing 1 + 0.9 + ... + 0.1:

```
double item = 1; double sum = 0;
while (item != 0) { // No guarantee
  sum += item;
  item -= 0.1;
}
System.out.println(sum);
```

- This loop seems ok on the surface, but can become an infinite loop due to approximations in the floating-point arithmetic

do-while & for Loops

```
do {
  // Loop body;
  Statement(s);
} while (loop-continuation-condition);
```

```
for (initial-action; loop-continuation-
condition; action-after-each-iteration) {
  // loop body;
  Statement(s);
}
```

```
int i;
for (i = 0; i < 100; i++) {
  System.out.println("Welcome to Java!");
}
```

Note

- The initial-action in a for loop – a list of zero or more comma-separated expressions
- The action-after-each-iteration in a for loop – a list of zero or more comma-separated statements

```
for (int i = 1; i < 100; System.out.println(i++));
for (int i = 0, j = 0; (i + j < 10); i++, j++) {
  // Do something
}
```

- The two for loops are correct
  - Rarely used in practice!

Note

- If the loop-continuation-condition in a for loop is omitted, it is implicitly true
- In case of infinite loops, it is better to use the equivalent while loop to avoid confusion

```
for ( ; ; ) {
  // Do something
}
```

Equivalent

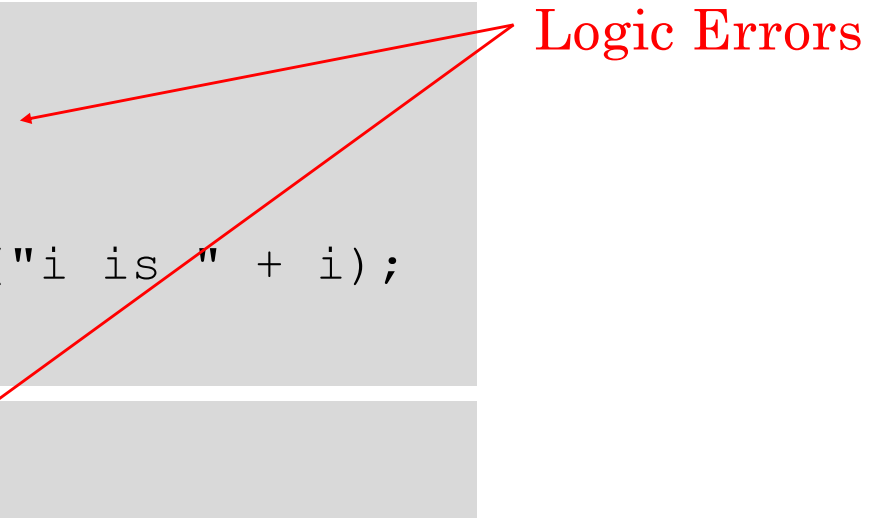
```
while (true) {
  // Do something
}
```

Caution

- Adding a semicolon at the end of the for clause before the loop body is a common mistake:

```
int i;
for (i=0; i<10; i++);
{
  System.out.println("i is " + i);
}
```

```
int i=0;
while (i < 10);
{
  System.out.println("i is " + i);
  i++;
}
```



Caution, cont.

- The semicolon is needed at the end of the do loop!

```
int i=0;
do {
  System.out.println("i is " + i);
  i++;
} while (i<10);
```

Correct!

Which Loop to Use?

- The three forms of loop statements (while, do-while, and for) are expressively equivalent:

```
while (cont-condition) {
  // Loop body
}
```

Equivalent

```
for ( ; cont-condition; ) {
  // Loop body
}
```

```
for (initial-action;
     cont-condition;
     action-after-iter) {
  // Loop body;
}
```

Equivalent

```
initial-action;
while (cont-condition) {
  // Loop body;
  action-after-iter;
}
```

Recommendations

- Use the one that is most intuitive and comfortable for you!
- In general:
  - ***for*** loop – a fixed number of repetitions (e.g., print a message 100 times)
  - ***while*** loop – unknown number of repetitions (e.g., reading the numbers until the input is 0)
  - ***do-while*** loop – replace a while loop if the loop body needs to be executed before testing the continuation condition

Minimizing Numerical Errors

- Numeric errors involving floating-point numbers are inevitable

```
public class TestSum {
  public static void main(String[] args) {
    // Initialize sum
    float sum = 0;

    // Add 0.01, 0.02, ..., 0.99, 1 to sum
    for (float i = 0.01f; i <= 1.0f; i += 0.01f)
      sum += i;

    // Display result
    System.out.println("The sum is " + sum);
  }
}
```

50.499985 😱

```
for (var i = 0.01d; i <= 1.0d; i += 0.01d)
```

49.500004 😱

There is still hope

```
public static void main(String[] args) {
        // Initialize sum
        BigDecimal sum = new BigDecimal(BigInteger.ZERO);

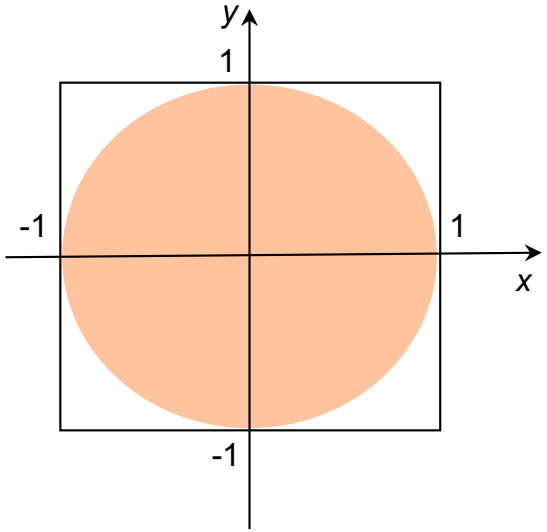
        // Add 0.01, 0.02, ..., 0.99, 1 to sum
        for (var i = 0.01d; i <= 1.0d; i += 0.01d)
            sum = sum.add(new BigDecimal(i),
               MathContext.DECIMAL64);

        // Display result
        System.out.println("The sum is " + sum);
    }
```

Float <-> DECIMAL32

Problem: *Monte Carlo Simulation*

- Refers to a technique that uses random numbers and probability to solve problems
- Wide range of applications in computational mathematics, physics, chemistry, and finance
- Use case: Monto Carlo simulation for estimating π



circleArea / squareArea = π / 4

π ≈ 4 * hits / 1.000.000

Using break and continue

```
int number=0;
w1: while (number < 20) {
  number++;
  sum += number;
  if (sum >= 100) break w1;
}
```

```
int number=0;
while (number < 20) {
  number++;
  if (number == 10 || number == 11)
    continue;
  sum += number;
}
```